



Haute Ecole de la Province de Liège

Techniques avancées de gestion des données

Bases de données avancées et XML

Leçon 3 : Schémas XML (XSD)

Samuel Hiard

Année académique 2023-2024

Agenda

- XSD
 - Besoin et avantages
 - Comparaison avec DTD
 - Éléments simples
 - Éléments complexes
 - Restrictions
 - Attributs
 - Entités
 - Portée des déclarations
 - Types de données
 - XSD et namespaces

Agenda

- XSD
 - Besoin et avantages
 - Comparaison avec DTD
 - Éléments simples
 - Éléments complexes
 - Restrictions
 - Attributs
 - Entités
 - Portée des déclarations
 - Types de données
 - XSD et namespaces

Besoins et avantages

- XSD (XML Schema Definition) est une recommandation du W3C
- Nous avons déjà vu la problématique des namespaces avec le DTD. XSD apporte une réponse à ce problème.
- XSD permet aussi :
 - De définir un schéma à l'aide d'XML
 - pas besoin d'apprendre un nouveau langage
 - On peut parser un document schéma à l'aide d'un parser XML
 - De contraindre la valeur des champs
 - Ce que DTD ne peut pas faire, car tout est #PCDATA

Agenda

- XSD
 - Besoin et avantages
 - Comparaison avec DTD
 - Éléments simples
 - Éléments complexes
 - Restrictions
 - Attributs
 - Entités
 - Portée des déclarations
 - Types de données
 - XSD et namespaces

Comparaison avec un DTD

DTD

```
<!ELEMENT note (to, from, heading, body)>
```

```
<!ELEMENT to (#PCDATA)>
```

```
<!ELEMENT from (#PCDATA)>
```

```
<!ELEMENT heading (#PCDATA)>
```

```
<!ELEMENT body (#PCDATA)>
```

Comparaison avec un DTD (2)

XSD

```
<xs:element name="note">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="to" type="xs:string"/>
      <xs:element name="from" type="xs:string"/>
      <xs:element name="heading" type="xs:string"/>
      <xs:element name="body" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Déclaration d'un XSD

- Nous commencerons d'abord par valider des éléments et attributs en dehors de tout namespace.
- L'inclusion dans un namespace rend les choses un peu plus complexe, ce pourquoi nous simplifions pour commencer.

- Déclaration du schema

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
...
</xs:schema>
```

- Dans le XML

```
<?xml version="1.0" encoding="UTF-8"?>
<root
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="myschema.xsd">
...
</root>
```

- `noNamespaceSchemaLocation` : car nos éléments et attributs sont en-dehors de tout namespace

Agenda

- XSD

- Besoin et avantages
- Comparaison avec DTD
- **Éléments simples**
- Éléments complexes
- Restrictions
- Attributs
- Entités
- Portée des déclarations
- Types de données
- XSD et namespaces

Éléments simples

- La déclaration d'un élément la plus simple est la suivante :

```
<xs:element name="nom" type="type" />
```

- où *nom* est le nom de l'élément, et *type*, son type.
- Le type peut être un des 44 types prédéfinis ou un type défini dans le schéma
 - Nous verrons certains types prédéfinis et comment définir son propre type un peu plus loin dans cette leçon
- Nous avons déjà rencontré les éléments simples. Par exemple :

```
<xs:element name="from" type="xs:string" />
```

Éléments simples (2)

- Il est aussi possible d'utiliser la balise `simpleType`

```
<xs:element name="xxx">  
  <xs:simpleType>...</xs:simpleType>  
</xs:element>
```

- Il faudra bien sûr définir le type (à la place des ...)
 - Voir plus loin car besoin de concepts comme « restriction », par ex.

Agenda

- XSD

- Besoin et avantages
- Comparaison avec DTD
- Éléments simples
- **Éléments complexes**
- Restrictions
- Attributs
- Entités
- Portée des déclarations
- Types de données
- XSD et namespaces

Éléments complexes

Comme vu précédemment avec le DTD, le contenu d'un élément complexe peut être de 5 types :

- Du texte simple (différent d'un contenu d'un élément simple)
 - Un ou plusieurs autres éléments enfants
 - Un mélange d'éléments et de texte
 - Rien. L'élément est vide.
 - N'importe quoi.
-
- On le définit avec `complexType`

Eléments complexes : texte uniquement

- L'élément complexe contient un élément simple.
- Il faut rajouter `extension` ou `restriction` au type de base
 - Nous verrons leur utilité un peu plus tard.

- Par exemple, le document XSD suivant

```
<xs:element name="shoesize">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:integer">
        </xs:extension>
      </xs:simpleContent>
    </xs:complexType>
  </xs:element>
```

- Rend valide ce document XML

```
<shoesize>35</shoesize>
```

Eléments enfants

- Comme pour le DTD, on peut déterminer autant d'enfants que l'on souhaite.
- L'ordre doit également être respecté
- On utilise alors `sequence`.

- Par exemple :

```
<xs:element name="person">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="firstname" type="xs:string"/>
      <xs:element name="lastname" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Éléments enfants : indicateurs

- Correspond aux *opérateurs* du DTD
- Permet d'indiquer un choix, avec `choice`
- Spécifie le nombre minimum et maximum d'occurrences avec `minOccurs` et `maxOccurs` (peut être infini avec `unbounded`)
- L'indicateur `all` permet aussi de spécifier que chaque élément doit apparaître, mais que l'ordre n'a pas d'importance.

Exemple complet : XML

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<persons xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:noNamespaceSchemaLocation="family.xsd">
```

```
<person>
```

```
  <full_name>Hege Refsnes</full_name>
```

```
  <child_name>Cecilie</child_name>
```

```
</person>
```

```
<person>
```

```
  <full_name>Tove Refsnes</full_name>
```

```
  <child_name>Hege</child_name>
```

```
  <child_name>Stale</child_name>
```

```
  <child_name>Jim</child_name>
```

```
  <child_name>Borge</child_name>
```

```
</person>
```

```
<person>
```

```
  <full_name>Stale Refsnes</full_name>
```

```
</person>
```

```
</persons>
```

Exemple complet : XSD

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
elementFormDefault="qualified">

  <xs:element name="persons">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="person" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="full_name" type="xs:string"/>
              <xs:element name="child_name" type="xs:string"
minOccurs="0" maxOccurs="5"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

Eléments mixtes

- Rappel : pour pouvoir valider du contenu comme celui-ci :

```
<book> Du <em>texte</em>, une <cite>citation</cite> et encore du  
<em>texte</em>.</book>
```

- Utilisation de l'attribut `mixed` dans la définition du type complexe

- Par exemple :

```
<xs:element name="book">  
  <xs:complexType mixed="true">  
    <xs:choice minOccurs="0" maxOccurs="unbounded">  
      <xs:element name="em" type="xs:string" />  
      <xs:element name="cite" type="xs:string" />  
    </xs:choice>  
  </xs:complexType>  
</xs:element>
```

- Notez que nous avons dû utiliser `choice` au lieu de `sequence`, sinon les tags `em` et `cite` devraient apparaître seulement une fois, et dans cet ordre.

Éléments vides

- Il existe plusieurs méthodes pour déclarer un élément vide
 - Définir un type simple constitué d'une chaîne de caractères avec une taille maximum de 0
 - Imposer un élément fixe vide
 - ...
- Le plus simple étant encore de déclarer l'élément avec un type qui ne contient rien :

```
<xs:element name="myEmptyElement">  
  <xs:complexType/>  
</xs:element>
```

- A noter que la déclaration du `complexType` est indispensable. Sans elle, on pourrait alors mettre n'importe quoi dans `myEmptyElement`.

Élément quelconque

- Utilisation de `any`
- Possibilité de remplacer ceci par n'importe quel élément (éventuellement défini dans un autre XSD)
- Rend XML extensible !

- Exemple:

```
<xs:element name="person">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="firstname" type="xs:string"/>
      <xs:element name="lastname" type="xs:string"/>
      <xs:any minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Agenda

- **XSD**

- Besoin et avantages
- Comparaison avec DTD
- Éléments simples
- Éléments complexes
- **Restrictions**
- Attributs
- Entités
- Portée des déclarations
- Types de données
- XSD et namespaces

Restrictions/facettes

- Possibilité d'appliquer des contraintes sur le contenu des éléments
- On utilise l'élément `xs:restriction` et son attribut `base` pour spécifier le type de base.
- Chaque contrainte est appliquée via une facette (*facet*), qui est un élément de `xs:restriction`
- Chaque facette dépend du type de base

- Utilisation :

```
<xs:restriction base="...">
    <!-- facets -->
</xs:restriction>
```

- Exemple :

```
<xs:element name="age">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="120"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Restrictions et types de données

- Il y en a vraiment beaucoup. Toutefois les plus courants sont :

Contrainte	Description
enumeration	Définit une liste de valeurs acceptables
fractionDigits	Indique le nombre maximal de décimales admis. ≥ 0
length	Indique le nombre exact de caractères ou liste des articles autorisés. ≥ 0
maxExclusive	Indique les limites supérieures pour les valeurs numériques (la valeur doit être inférieure à cette valeur)
maxInclusive	Indique les limites supérieures pour les valeurs numériques (la valeur doit être inférieure ou égale à cette valeur)
maxLength	Indique le nombre maximal de caractères ou liste des articles autorisés. ≥ 0
minExclusive	Indique les limites inférieures de valeurs numériques (la valeur doit être supérieure à cette valeur)
minInclusive	Indique les limites inférieures de valeurs numériques (la valeur doit être supérieure ou égale à cette valeur)
minLength	Indique le nombre minimum de caractères ou liste des articles autorisés. ≥ 0
pattern	Définit la séquence exacte des caractères qui sont acceptables
totalDigits	Indique le nombre exact de chiffres autorisés. > 0
whiteSpace	Indique comment l'espace blanc (sauts de ligne, des onglets, des espaces et des retours chariot) est manipulé

Valeur par défaut et valeur fixe

- Lorsque le type est simple, il est possible de donner une valeur par défaut ou une valeur fixe à l'élément comme dans les deux exemples suivants.
- Il faut pour cela donner des valeurs aux attributs `default` ou `fixed` de l'élément `xs:element`.

```
<xs:element name="title" type="xs:string" default="Titre par défaut" />
```

```
<xs:element name="title" type="xs:string" fixed="Titre fixe" />
```

- Notez que cela s'applique aussi aux attributs

Agenda

- **XSD**

- Besoin et avantages
- Comparaison avec DTD
- Éléments simples
- Éléments complexes
- Restrictions
- **Attributs**
- Entités
- Portée des déclarations
- Types de données
- XSD et namespaces

Attributs

- Les attributs se déclarent comme des éléments.

```
<xs:attribute name="lang" type="xs:string"/>
```

- Il est possible de les intégrer dans des éléments, qu'ils soient vides ou non
- Exemples :

extension

```
<xs:complexType name="book">  
  <xs:simpleContent>  
    <xs:extension base="xs:string">  
      <xs:attribute name="lang"  
        type="xs:string"/>  
    </xs:extension>  
  </xs:simpleContent>  
</xs:complexType>
```

élément vide

```
<xs:element name="order">  
  <xs:complexType>  
    <xs:attribute name="num"  
      type="xs:int" />  
  </xs:complexType>  
</xs:element>
```

Combiner extension et restriction

- Il est possible d'ajouter des restrictions sur les attributs (comme sur les éléments).
- Pour restreindre un contenu, il faut le déclarer dans `restriction`, mais pour déclarer un attribut, il faut être dans `extension`.
- L'exemple suivant ne fonctionnera pas :

```
<xs:complexType name="ValueWithUnits">
  <xs:simpleContent>
    <xs:extension base="xs:double">
      <xs:attribute name="uom" fixed="second"/>
      <xs:minInclusive="0"/>
      <xs:maxInclusive="10"/>
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
```

Combiner extension et restriction (2)

- Il faut alors déclarer un type intermédiaire et le réutiliser dans l'autre

```
<xs:simpleType name="RestrictedDouble">
  <xs:restriction base="xs:double">
    <xs:minInclusive value="0" />
    <xs:maxInclusive value="10" />
  </xs:restriction>
</xs:simpleType>
<xs:complexType name="ValueWithUnits">
  <xs:simpleContent>
    <xs:extension base="RestrictedDouble">
      <xs:attribute name="uom" fixed="second" />
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
```

Attributs d'attributs

- Comme nous l'avons déjà vu, il est possible de définir une valeur par défaut et d'imposer un contenu fixe à un attribut (comme pour un élément).
- Il suffit d'utiliser l'attribut `default` ou `fixed` dans la déclaration de l'attribut.
- Un attribut est, par défaut, optionnel, mais on peut le rendre obligatoire avec `required`

```
<xs:attribute name="lang" type="xs:string" use="required"/>
```

Agenda

- **XSD**

- Besoin et avantages
- Comparaison avec DTD
- Éléments simples
- Éléments complexes
- Restrictions
- Attributs
- **Entités**
- Portée des déclarations
- Types de données
- XSD et namespaces

Entités

- Il n'est pas possible de définir des entités dans un XSD, cela doit se faire dans un DTD

- `xs:entity` existe, mais uniquement pour faire référence à une entité déclarée dans un DTD

- Exemple :

```
<xs:element name="picture">
  <xs:complexType>
    <xs:attribute name="location" type="xs:ENTITY"/>
  </xs:complexType>
</xs:element>
```

```
<!DOCTYPE catalog SYSTEM "catalog.dtd" [
  <!NOTATION jpeg SYSTEM "JPG">
  <!ENTITY prod557 SYSTEM "prod557.jpg" NDATA jpeg>
  <!ENTITY prod563 SYSTEM "prod563.jpg" NDATA jpeg>
]>
```

```
<catalog>
  <product>
    <number>557</number>
    <picture location="prod557"/>
  </product>
</catalog>
```


Groupes

- Il est possible de définir des groupes, que l'on peut réutiliser plus tard
- Un peu comme les entités paramètres dans le DTD

- Exemple :

```
<xs:group name="persongroup">
  <xs:sequence>
    <xs:element name="firstname" type="xs:string"/>
    <xs:element name="lastname" type="xs:string"/>
  </xs:sequence>
</xs:group>
```

```
<xs:element name="person" type="personinfo"/>
```

```
<xs:complexType name="personinfo">
  <xs:sequence>
    <xs:group ref="persongroup"/>
    <xs:element name="country" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Agenda

- XSD

- Besoin et avantages
- Comparaison avec DTD
- Éléments simples
- Éléments complexes
- Restrictions
- Attributs
- Entités
- **Portée des déclarations**
- Types de données
- XSD et namespaces

Portée des déclarations

- Un composant peut être déclaré ou défini globalement ou localement.
 - S'il est *global*, il est nommé et enfant direct de `xs:schema`. On peut y faire référence de partout dans le document.
 - S'il est *local*, il est anonyme et sa portée est limitée au composant dans lequel il est déclaré.
- Exemple

Global

```
<xs:simpleType name="yyy">
...
</xs:simpleType>
<xs:element name="xxx" type="yyy"/>
```

Local

```
<xs:element name="xxx">
  <xs:simpleType>...</xs:simpleType>
</xs:element>
```

Agenda

- XSD

- Besoin et avantages
- Comparaison avec DTD
- Éléments simples
- Éléments complexes
- Restrictions
- Attributs
- Entités
- Portée des déclarations
- Types de données
- XSD et namespaces

Type de données

- Il existe 44 types de données prédéfinis.
- Les voir tous sortirait du cadre de ce cours
- Ils sont séparés en deux catégories :
 - Primitifs
 - Dérivés
- Primitifs : Types « de base », sans réelle restriction
 - string, double, date, integer, decimal, ...
- Dérivés : Types dérivés des types de base, où l'on a ajouté une ou plusieurs restrictions
 - NormalizedString : chaîne de caractère où on retire les espaces
 - positiveInteger : Seulement pour les entiers positifs
 - ...

Agenda

- XSD

- Besoin et avantages
- Comparaison avec DTD
- Éléments simples
- Éléments complexes
- Restrictions
- Attributs
- Entités
- Portée des déclarations
- Types de données
- XSD et namespaces

XSD et namespaces

- XSD permet une gestion plus fine des namespaces que DTD
- Schémas sans espace de nom
 - On ne spécifie pas le namespace par défaut (pas de `targetNamespace`)
 - Les éléments du fichier XML n'ont pas d'espace de noms (pas de « : »)
 - Il faut déclarer l'espace de nom des schemas (« `xs:element` », « `xs:string` », ...)
 - Ce que nous avons fait jusqu'ici
- Espace de noms cible
 - Spécifié avec l'attribut `targetNamespace` de l'élément `xs:schema`
 - L'attribut `elementFormDefault` de l'élément `xs:schema` détermine quels éléments sont définis dans l'espace de nom
 - `Unqualified` : Seuls les éléments globaux sont dans l'espace de nom
 - `Qualified` : Tous les éléments sont dans l'espace de nom

Exemple Unqualified

```
<?xml version="1.0" encoding="iso-8859-1"?>
<!-- unqualified est la valeur par défaut de elementFormDefault -->
<xsd:schema targetNamespace="http://www.omega-one.org/~carton/"
elementFormDefault="unqualified"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="name">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="firstname" type="xsd:string"/>
        <xsd:element name="lastname" type="xsd:string"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

- Le document suivant est valide pour le schéma précédent.

```
<?xml version="1.0" encoding="iso-8859-1" standalone="no"?>
<tns:name xmlns:tns="http://www.omega-one.org/~carton/">
  <firstname>Gaston</firstname>
  <lastname>Lagaffe</lastname>
</tns:name>
```


Exemple qualified

```
<?xml version="1.0" encoding="iso-8859-1"?>
<xsd:schema targetNamespace="http://www.omega-one.org/~carton/"
elementFormDefault="qualified"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<xsd:element name="name">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="firstname" type="xsd:string"/>
      <xsd:element name="lastname" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
</xsd:schema>
```

- Le document suivant est valide pour le schéma précédent.

```
<?xml version="1.0" encoding="iso-8859-1" standalone="no"?>
<tns:name xmlns:tns="http://www.omega-one.org/~carton/">
  <tns:firstname>Gaston</tns:firstname>
  <tns:lastname>Lagaffe</tns:lastname>
</tns:name>
```

XSD et namespaces (2)

- Il est également possible de n'imposer un espace de nom que sur certains éléments
 - Form=« `qualified` » comme attribut du composant `xs:element`
- Il est aussi possible de spécifier un espace de nom pour les attributs
 - `attributeFormDefault`